attacks on their web application using automated SQL injection tools to determine if there are any vulnerabilities.

The following are some preventative security measures that website owners can take to avoid SQL injection attacks.

- **Validate all user inputs** - Make sure all user-submitted inputs are checked for special characters like "" that could be given as SQL query arguments. For instance, in the email and name fields, any special characters should be properly sanitised and filtered, and the phone number field should only contain digits, and so on.
- **Use Parameterized Queries and Stored Procedures** - Using Stored Procedures, which automatically offer an extra layer of protection to the database, is a good idea. The benefit of using a Saved Procedure is that the SQL code is created and stored in the database server first, and then invoked from the web application. When you utilise a stored procedure, the web application treats user input as data to be processed rather than SQL code to be executed.
- **Frequent updates and patches** - Make sure to deploy patches and upgrades to your system (especially the SQL database) on a regular basis and stay as current as possible.
- **Limit the access privileges** - Unless absolutely necessary, never connect to your database with an account that has administrator-level access (root-access). Use a user account with fewer privileges so that the hacker has less access to your system and the scope of the harm, if any, is limited.
- **Disable shell access completely** - Hackers can easily take control of your system using this functionality.
- **Ensure the safety of credentials** - Make sure your application and database passwords, as well as any other sensitive data, are encrypted and secure.

2. Cross-Site Scripting (XSS)

Cross-site scripting is a website attack tactic that uses a sort of injection to insert harmful scripts into otherwise productive and trustworthy websites. In most cases, the procedure entails delivering a malicious browser-side script to a different user. This is a frequent online application security problem that can occur at any point in the programme where input from the browser is received and utilised to generate output without first verifying or encrypting the data.

Let's say there's a name parameter in the URL - example.com/profile. *https://example.com/profile?user=Rakesh* would be the URL for the request.